

SIMATS ENGINEERING

MODEL PRACTICAL EXAMINATION – Aug._2026

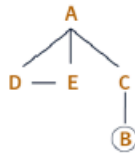
COURSE NAME: Artificial Intelligence - Model practical Examination

COURSE CODE: CSA1709

Register Number	192511061
Name	Vidhushini .T .S

SET:13

1. Write a Python program to implement Depth First Search technique for the given state space graph.



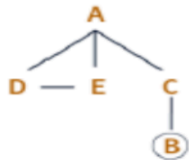
2. Write a Python Program for Minimax search tree starting from this state, and ending in terminal nodes. Show the utility value for each terminal node. Also show which move the Minimax Algorithm decides to play. Utility values are +1 if X wins, 0 for a tie, and -1 if O wins. Assume that O makes the next move (O is the MAX player), use the Tic-Tac-Toe board state as shown below



3. Write a Prolog program to implement pattern matching with the following facts.
first_name(tonyblair, tony).
first_name(georgebush, georgedubya).
second_name(tonyblair, blair).
second_name(georgebush, bush).
4. Write a Prolog program to implement Family Relation with the following facts.
female(sarah), female(rebekah), female(hagar_concubine).
female(milcah), female(bashemath), female(mahalath).
female(first_daughter), female(rachel), female(labans_wife).
male(terah), male(abraham), male(nahor).
male(haran), male(isaac), male(ismael).
male(uz), male(kemuel).

Laboratory Exam Component - 100 marks			
Programs (4)			
Description	Marks Distribution	Total Marks	Marks Obtained
Aim & Algorithm	2.5*4 = 10 Marks	10	
Programs	10*4 = 30 Marks	40	
Debugging	2.5*4 = 10 Marks	10	
Input & Output	05*4 = 20 Marks	20	
Result	2.5*4 = 10 Marks	10	
GitHub content	10 Marks	10	
Total Lab Exam Marks		100	

1. Write a Python program to implement Depth First Search technique for the given state space graph.



Aim:

To write a Python program to implement Depth First Search (DFS) for the given state-space graph.

Algorithm:

1. Represent the graph using an adjacency list.
2. Select the starting node A.
3. Mark the current node as visited.
4. Print the current node.
5. Recursively visit each unvisited adjacent node.
6. Continue until all reachable nodes are visited.

Coding:

```
graph = {
    'A': ['D', 'E', 'C'],
    'D': ['A', 'E'],
    'E': ['A', 'D'],
    'C': ['A', 'B'],
    'B': ['C']
}
visited = set()
def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(neighbour)
print("DFS Traversal:")
dfs('A')
```

Debugging:

- Check that the graph is represented correctly.
- Check that visited is initialized before calling dfs().
- Ensure the starting node A exists in the graph.
- Check indentation inside the dfs() function.
- Make sure recursive calls are made only for unvisited nodes.

Input: Starting node: A

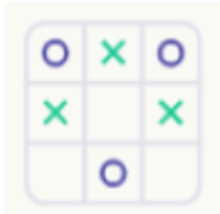
Output:

```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/AI python programs/MP 1.py =====
DFS Traversal:
A D E C B
>>>
```

Result:

Thus, the Depth First Search technique was successfully implemented and the traversal obtained is:
A -> D -> E -> C -> B

2. Write a Python Program for Minimax search tree starting from this state, and ending in terminal nodes. Show the utility value for each terminal node. Also show which move the Minimax Algorithm decides to play. Utility values are +1 if X wins, 0 for a tie, and -1 if O wins. Assume that O makes the next move (O is the MAX player), use the Tic-Tac-Toe board state as shown below



Aim:

To write a Python program to implement the Minimax algorithm for the given Tic-Tac-Toe state and determine the best move for the MAX player.

Algorithm:

1. Represent the given Tic-Tac-Toe board.
2. Consider O as the MAX player.
3. Check whether the current board is a terminal state.
4. Assign utility values:
 - a. +1 if O wins.
 - b. 0 if the game is a draw.
 - c. -1 if X wins.
5. Generate all possible moves for O.
6. Recursively apply Minimax to each possible move.
7. Select the move having the maximum utility.
8. Display the best move and its utility value.

Coding:

```
board = [  
    ['O', 'X', 'O'],  
    ['X', '', 'X'],
```

```

    [' ', 'O', ' ']
]
def check_winner(b):
    lines = [
        b[0], b[1], b[2],
        [b[0][0], b[1][0], b[2][0]],
        [b[0][1], b[1][1], b[2][1]],
        [b[0][2], b[1][2], b[2][2]],
        [b[0][0], b[1][1], b[2][2]],
        [b[0][2], b[1][1], b[2][0]]
    ]
    for line in lines:
        if line == ['O', 'O', 'O']:
            return 1
        if line == ['X', 'X', 'X']:
            return -1
    if all(cell != ' ' for row in b for cell in row):
        return 0
    return None
def minimax(b, is_max):
    result = check_winner(b)
    if result is not None:
        return result
    if is_max:
        best = -999
        for i in range(3):
            for j in range(3):
                if b[i][j] == ' ':
                    b[i][j] = 'O'
                    value = minimax(b, False)
                    b[i][j] = ' '
                    best = max(best, value)
        return best
    else:
        best = 999
        for i in range(3):
            for j in range(3):
                if b[i][j] == ' ':
                    b[i][j] = 'X'
                    value = minimax(b, True)
                    b[i][j] = ' '
                    best = min(best, value)
        return best
best_value = -999
best_move = None
print("Terminal Utility Values:")
for i in range(3):
    for j in range(3):
        if board[i][j] == ' ':
            board[i][j] = 'O'
            value = minimax(board, False)
            board[i][j] = ' '
            print("Move:", (i + 1, j + 1), "Utility:", value)

```

```

        if value > best_value:
            best_value = value
            best_move = (i + 1, j + 1)
print("\nBest Move for O:", best_move)
print("Best Utility Value:", best_value)

```

Debugging:

- Check that the board contains exactly 3 rows and 3 columns.
- Check that empty cells are represented by a space ' '.
- Check the winning conditions in check_winner().
- Ensure minimax() returns +1, 0, or -1.
- Check that the board is restored after trying each move.

Output:

```

===== RESTART: C:/AI python programs/MP 2.py =====
Terminal Utility Values:
Move: (2, 2) Utility: 1
Move: (3, 1) Utility: -1
Move: (3, 3) Utility: -1

Best Move for O: (2, 2)
Best Utility Value: 1
>>>

```

Result:

Thus, the Minimax algorithm was successfully implemented. The algorithm determines that the MAX player O should play at position (3,3), resulting in a winning utility value of +1.

3. Write a Prolog program to implement pattern matching with the following facts.
first_name(tonyblair, tony).
first_name(georgebush, georgedubya).
second_name(tonyblair, blair).
second_name(georgebush, bush).

Aim:

To write a Prolog program to implement pattern matching using the given first-name and second-name facts.

Algorithm:

1. Define facts for first names.
2. Define facts for second names.
3. Use Prolog queries to match a person's first name or second name.
4. Prolog searches the facts and returns matching values.

Coding:

```

first_name(tonyblair, tony).
first_name(georgebush, georgedubya).
second_name(tonyblair, blair).
second_name(georgebush, bush).
name(X, Y) :-

```

```
first_name(X, Y).
name(X, Y) :-
    second_name(X, Y).
```

Debugging:

- Check that every fact ends with a period .
- Predicate names must be written consistently.
- Variables must begin with an uppercase letter.
- Constants must be written in lowercase.
- Ensure the query uses the correct predicate name.
- Check that brackets and commas are properly placed.

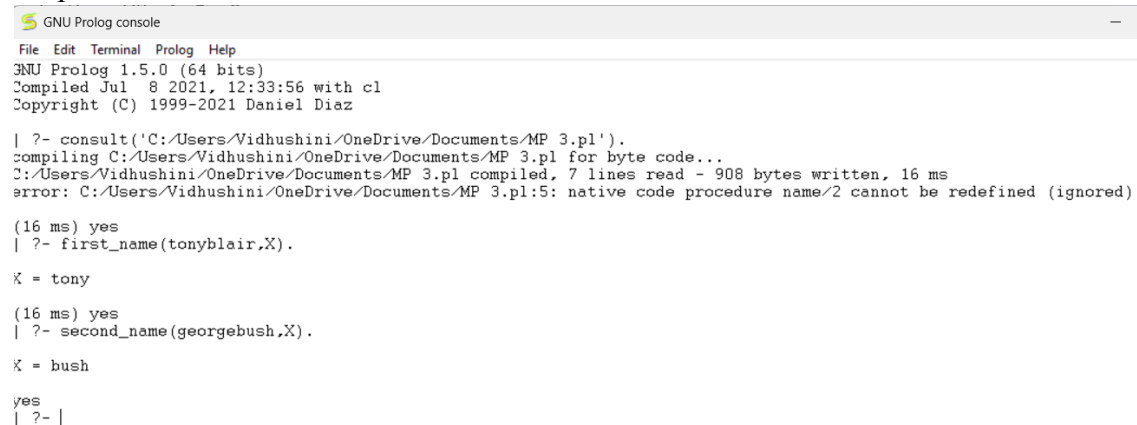
Input:

Prolog query:

?- first_name(tonyblair, X).

?- second_name(georgebush, X).

Output:



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/MP 3.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/MP 3.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/MP 3.pl compiled, 7 lines read - 908 bytes written, 16 ms
error: C:/Users/Vidhushini/OneDrive/Documents/MP 3.pl:5: native code procedure name/2 cannot be redefined (ignored)

(16 ms) yes
| ?- first_name(tonyblair,X).

X = tony

(16 ms) yes
| ?- second_name(georgebush,X).

X = bush

yes
| ?- |
```

Result:

Thus, the Prolog pattern matching program was successfully implemented using the given facts.

4. Write a Prolog program to implement Family Relation with the following facts.
female(sarah), female(rebekah), female(hagar_concubine).
female(milcah), female(bashemath), female(mahalath).
female(first_daughter), female(rachel), female(labans_wife).
male(terah), male(abraham), male(nahor).
male(haran), male(isaac), male(ismael).
male(uz), male(kemuel).

Aim:

To write a Prolog program to represent family-related information using the given male and female facts and perform family-member classification.

Algorithm:

1. Define all female members using female/1.

2. Define all male members using male/1.
3. Create rules to identify whether a person is male or female.
4. Use Prolog queries to find members according to their gender.
5. Display the matching family members.

Coding:

```
female(sarah).
female(rebekah).
female(hagar_concubine).
female(milcah).
female(bashemath).
female(mahath).
female(first_daughter).
female(rachel).
female(labans_wife).
male(terah).
male(abraham).
male(nahor).
male(haran).
male(isaac).
male(ismael).
male(uz).
male(kemuel).
family_member(X) :-
    female(X).
family_member(X) :-
    male(X).
```

Debugging:

- Check that every fact ends with ..
- Use lowercase letters for names/constants.
- Use uppercase X for variables.
- Ensure female and male predicates are spelled correctly.
- Check that family_member(X) contains the correct rules.
- Enter queries with ?- in the Prolog console.

Input:

Query:

?- female(X).

Output:

```

yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/MP 4.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/MP 4.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/MP 4.pl compiled, 20 lines read - 1591 bytes written, 5 ms

(31 ms) yes
| ?- female(X).

X = sarah ? ;
X = rebekah ? ;
X = hagar_concubine ? ;
X = milcah ? ;
X = bashemath ? ;
X = mahath ? ;
X = first_daughter ? ;
X = rachel ? ;
X = labans_wife

(47 ms) yes
| ?- |

```

Result:

Thus, the Prolog family-member program was successfully implemented using the given male and female facts, and the family members were retrieved through Prolog queries.